

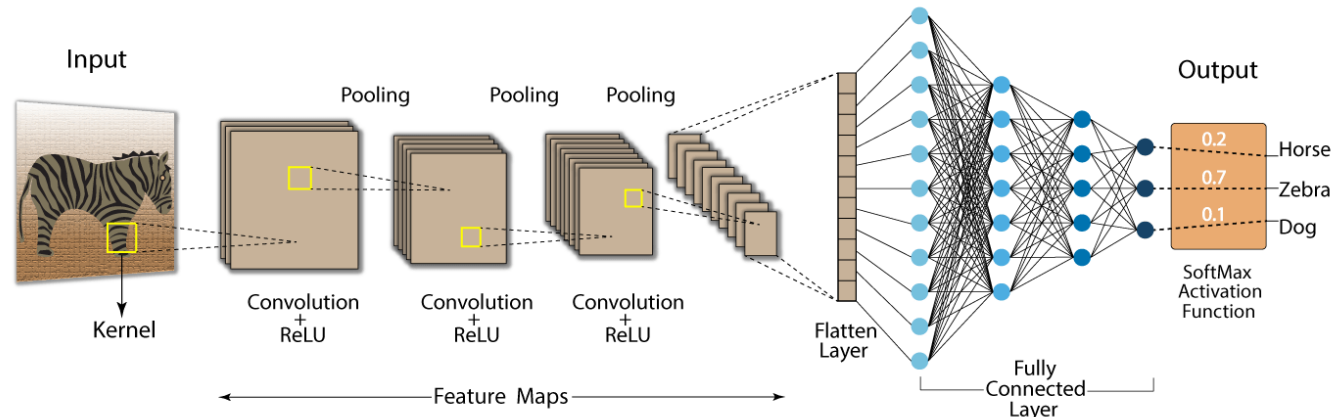


Module 5

Convolutional Neural Networks



DL: CNNs Pre-Trained Models



Review: Convolutions

- Below is a (3×3) kernel (convolution) applied to (5×5) image
- Stride is $(1,1)$

7	2	3	3	8
4	5	3	8	4
3	3	2	8	4
2	8	7	2	7
5	4	4	5	4

*

1	0	-1
1	0	-1
1	0	-1

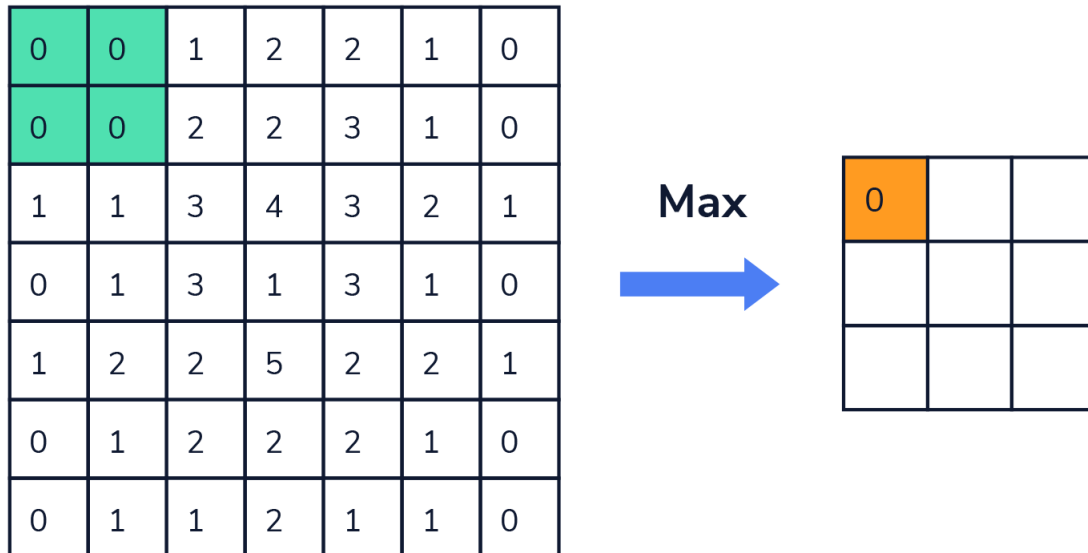
=

6		

$7 \times 1 + 4 \times 1 + 3 \times 1 +$
 $2 \times 0 + 5 \times 0 + 3 \times 0 +$
 $3 \times -1 + 3 \times -1 + 2 \times -1$
 $= 6$

Review: Pooling

- ❑ A pooling filter is used to downsample the images
- ❑ min, max and average pooling are the most used.



Padding

- ❑ Used to preserve the spatial dimensions of the input image
- ❑ Example: adding an extra layer of padding, e.g., padding = 1

0	0	0	0	0	0	0	0
0	3	3	4	4	7	0	0
0	9	7	6	5	8	2	0
0	6	5	5	6	9	2	0
0	7	1	3	2	7	8	0
0	0	3	7	1	8	3	0
0	4	0	4	3	2	2	0
0	0	0	0	0	0	0	0

$6 \times 6 \rightarrow 8 \times 8$

*

1	0	-1
1	0	-1
1	0	-1

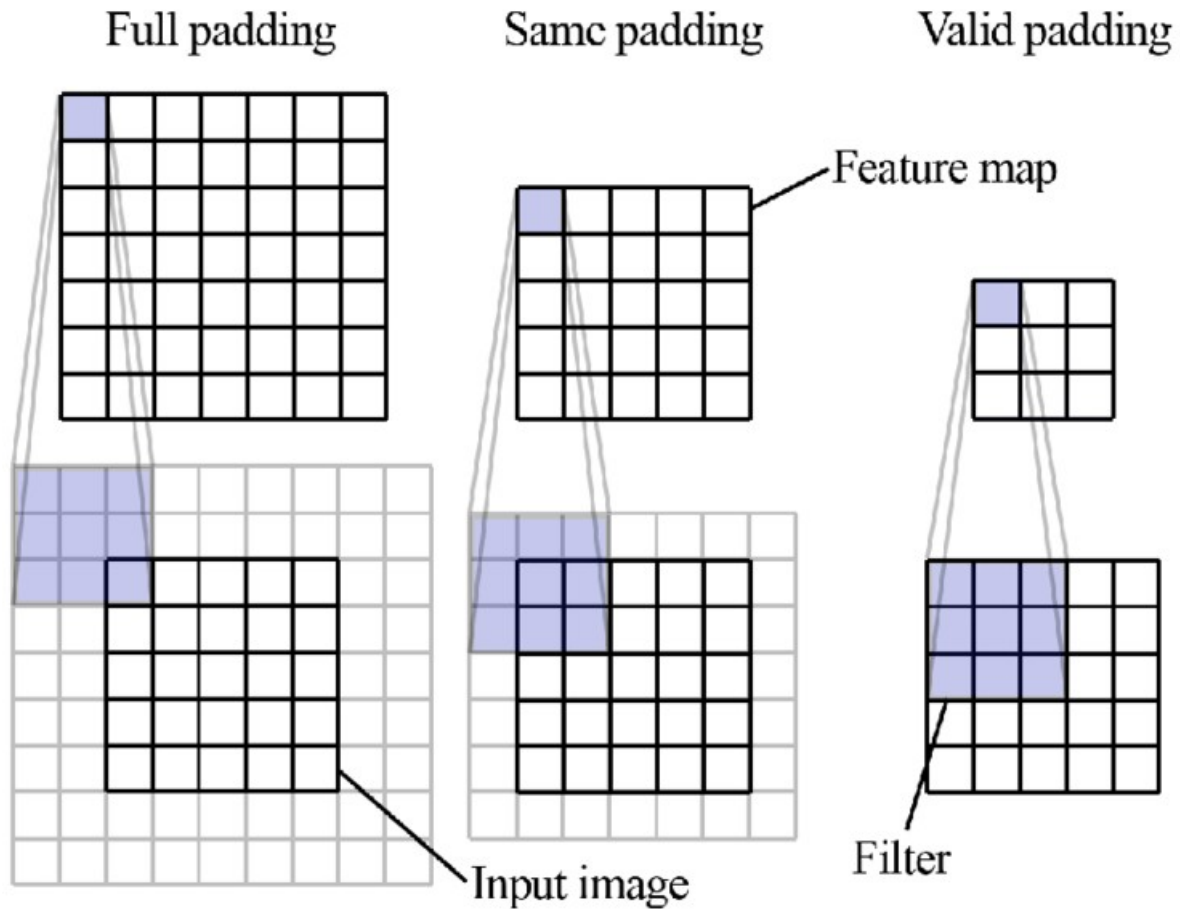
3×3

=

-10	-13	1			
-9	3	0			

6×6

Types of Padding



Output Size with Padding and Stride

SCSS

Copy code

```
output_size = (input_size - filter_size + 2*padding) / stride + 1  
              = (32 - 5 + 2*2) / 1 + 1  
              = 32
```

0	0	0	0	0	0	0	0
0	3	3	4	4	7	0	0
0	9	7	6	5	8	2	0
0	6	5	5	6	9	2	0
0	7	1	3	2	7	8	0
0	0	3	7	1	8	3	0
0	4	0	4	3	2	2	0
0	0	0	0	0	0	0	0

$6 \times 6 \rightarrow 8 \times 8$

*

1	0	-1
1	0	-1
1	0	-1

3×3

=

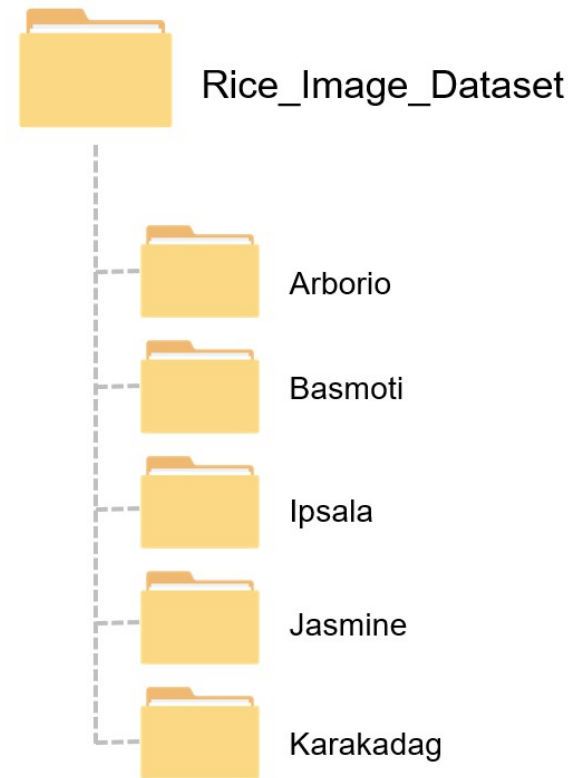
-10	-13	1			
-9	3	0			

6×6

Folder Structure for CNN Training

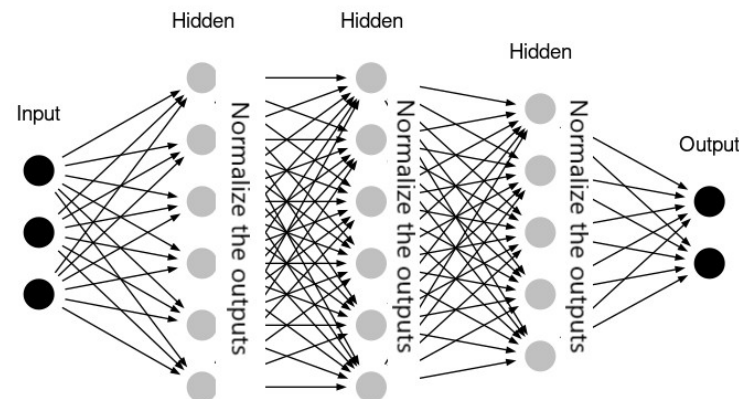
- ❑ A typical folder structure for a CNN-based image classification project:

```
cnnp_project/  
├── data/  
│   ├── train/  
│   │   ├── class_1/  
│   │   └── class_2/  
│   └── val/  
│       ├── class_1/  
│       └── class_2/  
├── models/  
│   └── saved_model.h5  
└── notebooks/  
    └── exploratory_analysis.ipynb
```



Batch Normalization

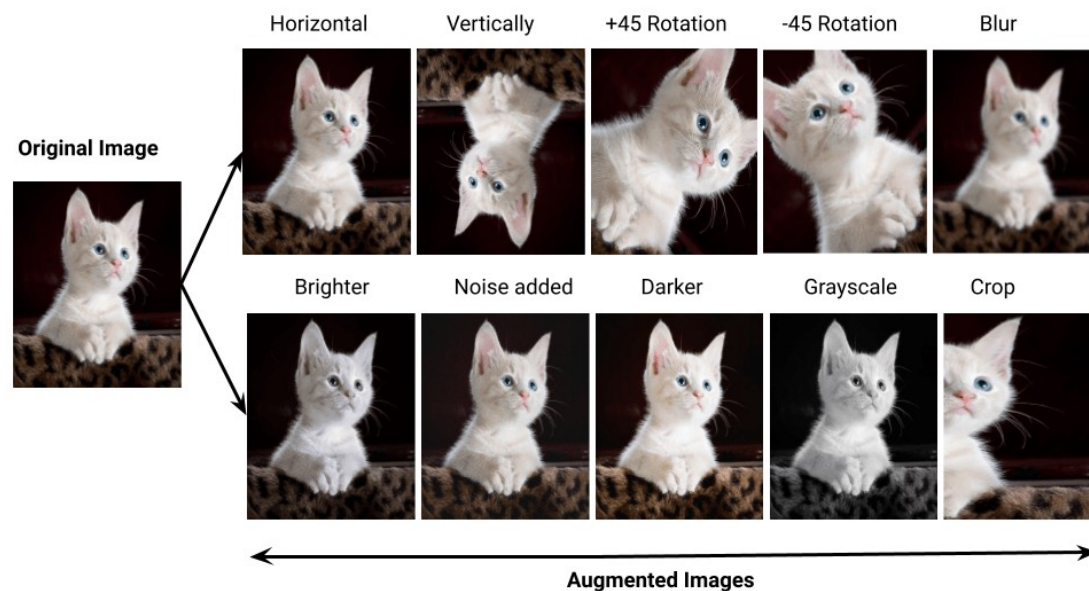
- ❑ Batch normalization is a technique used to improve the training of deep neural networks.
 - Normalizes the input of each layer to have zero mean and unit variance.
 - Helps reduce internal covariate shift.
 - Speeds up training and allows for higher learning rates.
 - Acts as a form of regularization, sometimes reducing the need for dropout.
 - Typically applied after linear transformation (e.g., convolution) and before the activation function.



Data Augmentation

❑ Data augmentation is a technique used to increase the size and diversity of training datasets.

- Image rotation, flipping (horizontal/vertical)
- Cropping and resizing
- Brightness and contrast adjustments
- Adding noise
- Translation and scaling
- Cutout and mixup methods

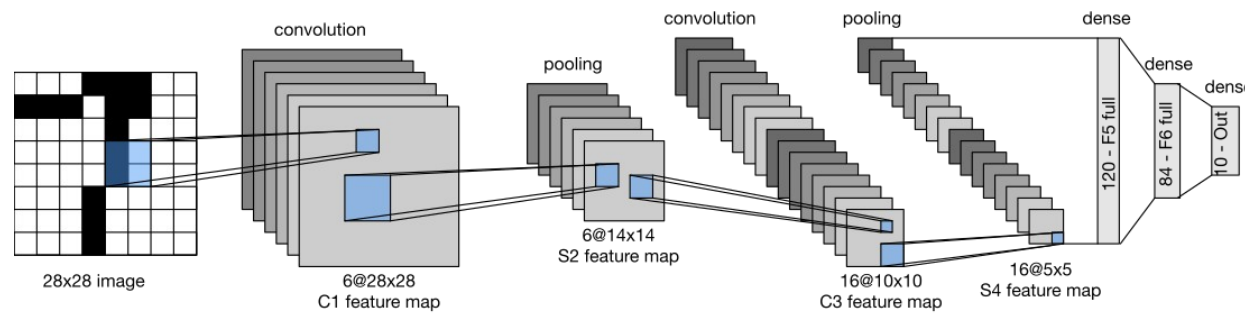




Python

Examples of CNN Architectures

- ❑ LeNet-5 is a foundational CNN architecture designed for handwritten digit recognition (e.g., MNIST).



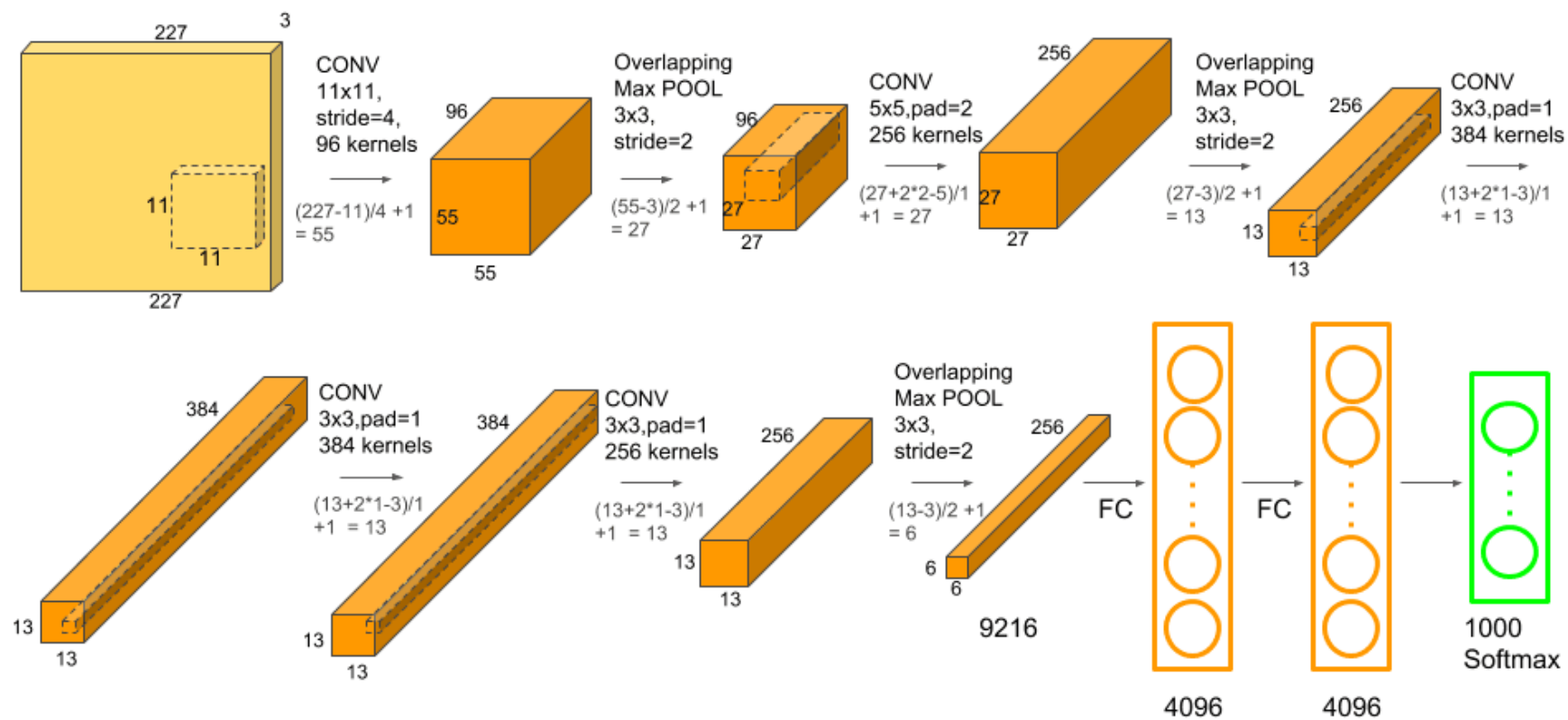
Architecture Overview:

1. Input: 32x32 grayscale image
2. C1 - Convolutional layer: 6 filters of size 5x5 → Output: 28x28x6
3. S2 - Downsampling (Avg Pooling): 2x2 → Output: 14x14x6
4. C3 - Convolutional layer: 16 filters → Output: 10x10x16
5. S4 - Downsampling (Avg Pooling): 2x2 → Output: 5x5x16
6. C5 - Fully connected convolution: Output: 120
7. F6 - Fully connected: Output: 84
8. Output Layer: Fully connected with 10 units (for classification)

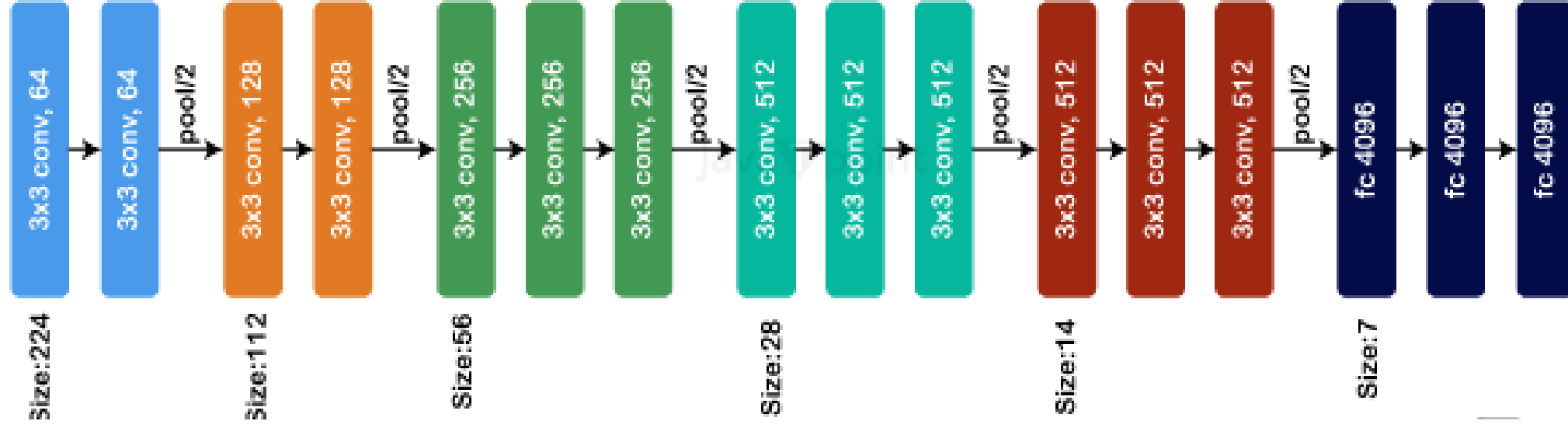
Examples of CNN Architectures

- ❑ AlexNet (2012): Introduced ReLU activation, dropout, and GPU training. Won ImageNet 2012.
- ❑ VGGNet (2014): Uses very small (3x3) filters and a deep architecture (16-19 layers).
- ❑ GoogLeNet (Inception) (2014): Introduced Inception modules to reduce parameters while maintaining depth.
- ❑ ResNet (2015): Introduced residual connections to enable very deep networks (e.g., 50,101 layers).
- ❑ DenseNet (2016): Each layer receives input from all previous layers (dense connectivity).
- ❑ MobileNet (2017): Lightweight CNN for mobile and embedded vision applications.

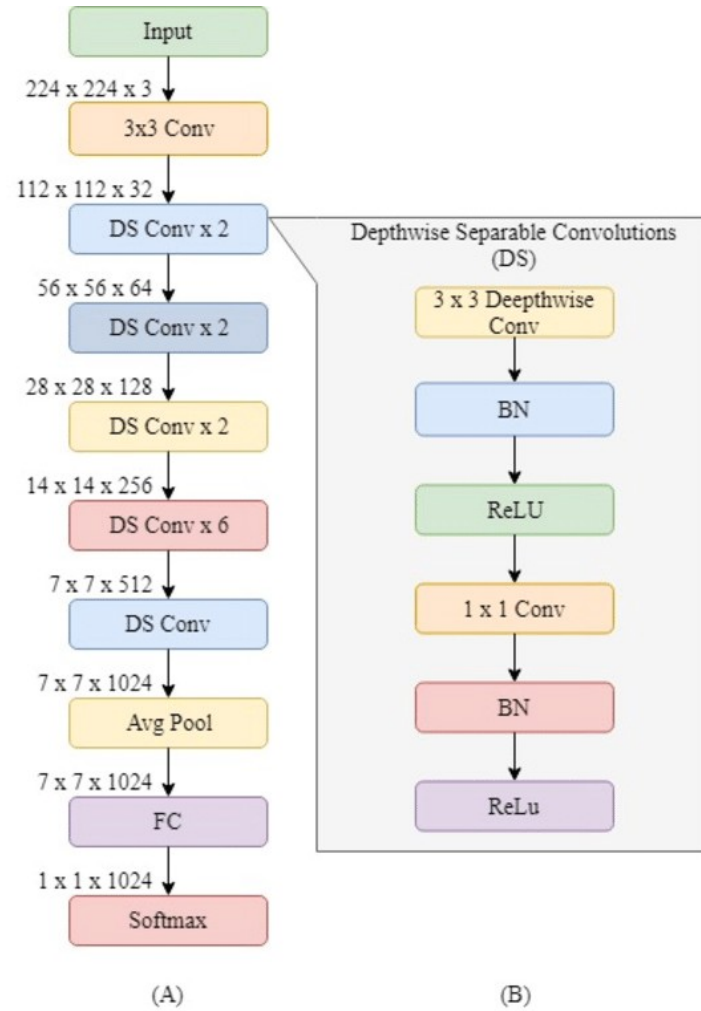
AlexNet



VGGNet



VGGNet





Python